TC ID	Test Objective	Precondition	Steps	Test Data	Result	Post-condition	Pass/ Fail
TC-001	Verify keyword generation from user input	Service initialized, valid user input exists	1. Call generate_keywords() 2. Pass user input text 3. Validate output	Input: "I need a fitness app for tracking workouts"	System returned: ["fitness", "app", "tracking", "workouts", "gym", "exercise"]	Keywords stored in database	**Pass**
TC-002	Fetch Reddit posts by keywords	Reddit API credentials configured, valid keywords	1. Call fetch_reddit_posts() 2. Pass keyword list 3. Verify response	Keywords: ["productivity", "app"]	System returned 280 Reddit posts, each containing: • Title • Post body • num of comments • Subreddit name	Posts cached in database	**Pass**
TC-003	Generate embeddings for text	Sentence transformer model loaded	1. Call generate_embedding() 2. Pass text string 3. Verify vector output	Text: "AI-powered fitness tracker"	Returns 384-dimensional vector	Embedding cached	**Pass**
TC-004	Filter semantically similar posts	Embeddings generated for posts	1. Call filter_similar_posts() 2. Set similarity threshold 3. Verify filtering	Threshold: 0.8, 50 posts	Returns posts above similarity threshold. E.g returns 100 posts above 0.5 threshold out of 280 posts	Filtered posts marked	**Pass**
TC-005	Cluster posts by topic	Embeddings available for all posts	1. Call cluster_posts() 2. Specify number of clusters 3. Verify clustering	100 posts, 5 clusters	Posts grouped into 5 distinct clusters	Cluster labels assigned	**Pass**
TC-006	Extract pain points from text	LLM service available, text data ready	1. Call extract_pain_points() 2. Pass post content 3. Verify extraction	Post: "I'm frustrated with expensive gym memberships"	Extracts: "High cost of gym memberships"	Pain point stored	**Pass**

TC-007	Rank pain points by severity	Pain points extracted and scored	1. Call rank_pain_points() 2. Apply ranking algorithm 3. Verify order	20 pain points with scores	Pain points ranked from highest to lowest severity	Rankings stored	**Pass**
TC-008	Combined search across sources	All APIs configured	1. Call search_similar_products() 2. Aggregate results 3. Verify completeness	Keywords: ["budget", "finance"]	Returns combined results from all sources	Complete market view	**Pass**
TC-009	Evaluate market demand	LLM API available, idea data ready	1. Call evaluate_market_demand() 2. Pass idea object 3. Verify score	Idea: AI fitness app	Returns score (1-5) with justifications	Score stored	**Pass**
TC-010	Evaluate feasibility	Team and resource data available	1. Call evaluate_feasibility() 2. Assess resources 3. Verify score	Team capabilities, timeline	Returns feasibility score (1-5)	Feasibility determined	**Pass**
TC-011	Orchestrate full validation pipeline	All services available, idea submitted	1. Call validate_idea() 2. Execute all validation steps 3. Verify completion	Complete idea object	Returns comprehensive validation report	Report stored in DB	**Pass**
TC-012	Create new idea	User authenticated, valid idea data	1. Call create_idea() 2. Pass idea details 3. Verify creation	Title, description, problem, solution	Idea created with unique ID	Idea stored in database	**Pass**
TC-013	Retrieve idea by ID	Idea exists in database	1. Call get_idea() 2. Pass idea ID 3. Verify retrieval	Valid idea ID	Returns complete idea object	No changes to data	**Pass**
TC-014	Generate validation report	Validation completed	1. Call generate_report()	Complete validation data	Returns structured report with all sections	Report ready for display	**Pass**

			2. Pass validation results 3. Verify report structure				
TC-015	Export report to PDF	Report generated	1. Call export_to_pdf() 2. Generate PDF 3. Verify formatting	Complete report data	Returns PDF file with proper formatting	PDF file created	**Pass**
TC-016	Export validation to PDF	Validation report available	1. Call export_validation_pdf() 2. Pass report data 3. Verify PDF creation	Complete validation report	PDF file created with all sections	PDF stored/downloadable	**Pass**
TC-017	User registration	Valid user data provided	1. Call signup() 2. Pass email, password 3. Verify user creation	Email: test@example.com, Password: SecurePass123!	User created, password hashed	User in database	**Pass**
TC-018	User login with valid credentials	User registered	1. Call login() 2. Pass email, password 3. Verify token	Valid credentials	Returns JWT access token	Token issued	**Pass**
TC-019	JWT token validation	Token issued	1. Make authenticated request 2. Verify token 3. Check expiration	Valid JWT token	Token validated, user identified	Access granted	**Pass**
TC-020	Establish database connection	MongoDB running, credentials configured	1. Call connect_to_database() 2. Verify connection 3. Check status	Valid connection string	Connection established successfully	Database accessible	**Pass**
TC-021	Submit valid idea	Form filled correctly	1. Fill all fields 2. Click submit 3. Verify API call	Valid idea data	Calls API with form data, shows success message	Idea submitted	**Pass**

TC-022	Display validation scores	Validation data available	1. Render component 2. Pass validation data 3. Verify score display	Complete validation report	Shows all metric scores (1-5) with visual indicators	Scores displayed	**Pass**
TC-023	Export report functionality	Report displayed	1. Click export button 2. Verify PDF generation 3. Check download	Complete report	Triggers PDF download	PDF exported	**Pass**
TC-024	Display multiple validations	Multiple validation results available	1. Render component 2. Pass validation array 3. Verify table	3 validation results	Shows table with all validations side-by-side	Comparison visible	**Pass**
TC-025	Display pain points list	Pain points data available	1. Render component 2. Pass pain points array 3. Verify display	10 pain points	Shows list with pain point text, severity, frequency	Pain points visible	**Pass**
TC-026	Allow authenticated access	User logged in	1. Render protected route 2. Verify access 3. Check content	Valid auth token	Renders protected content	Content accessible	**Pass**
TC-027	Make GET request	API service initialized	1. Call api.get() 2. Pass endpoint 3. Verify response	Endpoint: /api/ideas	Returns data with status 200	Data retrieved	**Pass**
TC-028	Make POST request	API service initialized	1. Call api.post() 2. Pass data 3. Verify response	Endpoint: /api/ideas, data: idea object	Returns created resource with status 201	Resource created	**Pass**
TC-029	Include auth token	User authenticated	1. Make authenticated request 2. Verify headers 3. Check token	Valid token	Request includes Authorization header	Token sent	**Pass**
TC-030	Login user	Hook initialized	1. Call login() 2. Pass credentials	Valid credentials	Updates user state, stores token	User logged in	**Pass**

			3. Verify state update				
TC-031	Logout user	User logged in	1. Call logout() 2. Verify state clear 3. Check token removal	Logged in user	Clears user state, removes token	User logged out	**Pass**
TC-032	Fetch user ideas	User authenticated	1. Call useIdeas() 2. Verify API call 3. Check data	User with ideas	Returns array of user's ideas	Ideas loaded	**Pass**
TC-033	Create new idea	Hook initialized	1. Call createIdea() 2. Pass idea data 3. Verify creation	New idea data	Creates idea, updates local state	Idea created	**Pass**
TC-034	Trigger validation	Idea exists	1. Call triggerValidation() 2. Pass idea ID 3. Verify API call	Valid idea ID	Initiates validation, returns validation ID	Validation started	**Pass**
TC-035	Fetch validation results	Validation complete	1. Call getValidation() 2. Pass validation ID 3. Verify results	Valid validation ID	Returns complete validation report	Results retrieved	**Pass**
TC-036	Provide auth state	Context provider mounted	1. Render provider 2. Access context 3. Verify state	Initial state	Provides user, token, isAuthenticated	State accessible	**Pass**
TC-037	Complete idea submission and validation	User registered and logged in	1. Submit new idea 2. Trigger validation 3. View results 4. Export report	Complete idea data	Idea created, validated, report generated and exported	Full workflow complete	**Pass**
TC-038	Frontend-Backend authentication	Both services running	1. Login from frontend 2. Verify token exchange 3. Make authenticated request	Valid credentials	Token received, stored, used in subsequent requests	Auth flow complete	**Pass**

TC-039	Idea CRUD operations	User authenticated	1. Create idea via frontend 2. Verify backend storage 3. Retrieve and display	Idea data	Idea created in DB, retrieved and displayed	CRUD operations work	**Pass**
TC-040	Real-time validation updates	Validation triggered	1. Start validation 2. Poll for updates 3. Display progress 4. Show final results	Idea for validation	Frontend receives progress updates, displays final report	Real-time updates work	**Pass**
TC-041	Create product for competitor analysis	User authenticated	1. Call create_product() 2. Pass product details 3. Verify creation	Product name: "TaskMaster Pro", description: "AI-powered task management", features: ["AI scheduling", "Team collaboration"]	Product created with unique ID	Product stored in database	**Pass**
TC-042	Start competitor analysis	Product exists in database	1. Call start_analysis() 2. Pass product ID 3. Verify analysis initiation	Valid product ID	Analysis initiated, returns analysis ID with HTTP 202	Analysis running in background	**Pass**
TC-043	Get competitor analysis results	Analysis completed	1. Call get_analysis() 2. Pass analysis ID 3. Verify results	Valid analysis ID	Returns top 5 competitors with features, pricing, URLs	Analysis results retrieved	**Pass**
TC-044	Generate feature comparison matrix	Competitor analysis complete	1. Extract features 2. Build comparison matrix 3. Verify structure	Product + 5 competitors	Returns matrix with features as rows, products as columns	Matrix ready for visualization	**Pass**

TC-045	Perform gap analysis	Competitor data available	1. Compare features 2. Identify gaps 3. Generate insights	Product features vs competitors	Returns opportunities, unique strengths, areas to improve, market gaps	Gap analysis complete	**Pass**
TC-046	Get analysis history for product	Multiple analyses exist	1. Call get_history() 2. Pass product ID 3. Verify list	Product with 3 analyses	Returns list of analyses sorted by date (newest first)	History retrieved	**Pass**
TC-047	Delete competitor analysis	Analysis exists	1. Call delete_analysis() 2. Pass analysis ID 3. Verify deletion	Valid analysis ID	Analysis deleted successfully	Analysis removed from database	**Pass**
TC-048	Update product details	Product exists	1. Call update_product() 2. Pass updated data 3. Verify update	Product ID + new description	Product updated successfully	Changes saved to database	**Pass**
TC-049	Get all user products	User has products	1. Call get_products() 2. Verify authentication 3. Check results	User with 5 products	Returns list of all user's products	Products retrieved	**Pass**
TC-050	Verify competitor discovery accuracy	Product description provided	1. Run analysis 2. Check competitor relevance 3. Verify sources	Product: "AI fitness tracker"	Returns relevant competitors from ProductHunt, Google search	Competitors validated	**Pass**
TC-051	Create launch plan	User authenticated, product data ready	1. Call create_plan() 2. Pass product details 3. Verify plan generation	Product name, description, target market, budget	Launch plan created with unique plan ID	Plan stored in database	**Pass**
TC-052	Get launch plan by ID	Plan exists in database	1. Call get_plan() 2. Pass plan ID 3. Verify retrieval	Valid plan ID	Returns complete launch plan with all sections	Plan retrieved	**Pass**

TC-053	Verify plan includes timeline	Launch plan generated	1. Check plan structure 2. Verify timeline section 3. Validate phases	Complete launch plan	Plan includes phased timeline with milestones	Timeline validated	**Pass**
TC-054	Verify plan includes go-to-market strategy	Launch plan generated	1. Check GTM section 2. Verify marketing channels 3. Validate customer acquisition	Complete launch plan	Plan includes GTM strategy with marketing channels, pricing, distribution	GTM strategy validated	**Pass**
TC-055	Verify plan includes budget allocation	Launch plan with budget data	1. Check budget section 2. Verify categories 3. Validate percentages	Total budget: $100,000	Plan includes budget breakdown: Product & Tech, Go-to-Market, Operations, Legal	Budget allocation validated	**Pass**
TC-056	Verify plan includes risk assessment	Launch plan generated	1. Check risk section 2. Verify identified risks 3. Validate mitigation strategies	Complete launch plan	Plan includes identified risks with mitigation strategies	Risk assessment validated	**Pass**
TC-057	Verify plan includes success metrics	Launch plan generated	1. Check metrics section 2. Verify KPIs 3. Validate measurement methods	Complete launch plan	Plan includes KPIs and success metrics	Metrics validated	**Pass**
TC-058	Get user's launch plan history	User has multiple plans	1. Call get_history() 2. Pass user ID 3. Verify sorting	User with 4 plans	Returns list of plans sorted by created_at (newest first)	History retrieved	**Pass**
TC-059	Verify LLM summarization quality	Market data available	1. Pass complex data to LLM 2. Generate summary 3. Verify coherence	Market research data, competitor analysis	Returns coherent, actionable summary	Summary validated	**Pass**

TC-060	Test plan generation performance	Service initialized	1. Start timer 2. Generate plan 3. Measure time	Standard product data	Plan generated within acceptable time (< 30 seconds)	Performance acceptable	**Pass**
TC-061	Submit problem discovery request	User authenticated	1. Call discover_problems() 2. Pass problem description 3. Verify request creation	Problem: "Finding affordable healthy meal options", Domain: "Food & Nutrition"	Request accepted with unique request ID	Request stored, processing initiated	**Pass**
TC-062	Get processing status	Request submitted	1. Call get_status() 2. Pass request ID 3. Verify status info	Valid request ID	Returns current stage, progress percentage, estimated time	Status retrieved	**Pass**
TC-063	Verify FAISS index creation	Embeddings generated	1. Check embeddings directory 2. Verify FAISS files 3. Test index loading	1000 post embeddings	FAISS index created: faiss_index.bin, metadata.json	Index ready for search	**Pass**
TC-064	Test HDBSCAN clustering	Filtered posts available	1. Call cluster_posts() 2. Apply HDBSCAN 3. Verify clusters	500 filtered posts	Posts clustered, noise identified, cluster labels assigned	Clustering complete	**Pass**
TC-065	Extract pain points from clusters	Clusters created	1. Call extract_pain_points() 2. Process each cluster 3. Verify extraction	15 clusters	Pain points extracted with titles, descriptions, post references	Pain points stored	**Pass**
TC-066	Rank pain points by relevance	Pain points extracted	1. Call rank_pain_points() 2. Apply ranking algorithm 3. Verify order	25 pain points, original query	Pain points ranked with relevance scores	Rankings stored	**Pass**
TC-067	Get pain points by domain	Pain points categorized	1. Call get_by_domain())	Valid input ID	Returns pain points grouped by business domain	Domain grouping complete	**Pass**

			2. Pass input ID 3. Verify grouping				
TC-068	Delete user input and associated data	Input exists with processed data	1. Call delete_input() 2. Pass input ID 3. Verify deletion	Valid input ID	Input and associated data deleted	Data removed from database	**Pass**
TC-069	Test processing lock mechanism	Multiple concurrent requests	1. Submit concurrent requests 2. Verify lock acquisition 3. Check rejection	2 requests for same input	First request acquires lock, second rejected with HTTP 409	Lock mechanism working	**Pass**
TC-070	Verify complete pipeline execution	All services available	1. Submit problem 2. Monitor stages 3. Verify completion	Complete problem description	Pipeline completes: keywords → Reddit → embeddings → filtering → clustering → pain points → ranking	Full pipeline successful	**Pass**
TC-071	Create idea with all fields	User authenticated	1. Call create_idea() 2. Pass complete data 3. Verify creation	Title, description, problem statement, solution, target market, business model, team capabilities	Idea created with all fields populated	Idea stored in database	**Pass**
TC-072	Update idea fields	Idea exists	1. Call update_idea() 2. Pass updated fields 3. Verify update	Idea ID + updated description and target market	Idea updated successfully	Changes saved to database	**Pass**
TC-073	Filter ideas by score range	Multiple ideas with validations	1. Call list_ideas() 2. Apply score filters 3. Verify results	min_score=3.5, max_score=5.0	Returns only ideas with scores in range	Filtered ideas retrieved	**Pass**
TC-074	Sort ideas by validation count	Ideas with varying validation counts	1. Call list_ideas() 2. Sort by validation_count 3. Verify order	sort_by=validation_count, sort_order=desc	Ideas sorted by number of validations (highest first)	Sorting applied	**Pass**

TC-075	Validate idea synchronously	Idea exists, LLM available	1. Call validate_idea() 2. Wait for completion 3. Verify results	Valid idea ID	Returns complete validation after 30-60 seconds	Validation complete	**Pass**
TC-076	Validate idea asynchronously	Idea exists	1. Call validate_async() 2. Verify immediate return 3. Poll for status	Valid idea ID	Returns validation ID immediately (HTTP 202)	Validation running in background	**Pass**
TC-077	Get validation status	Validation in progress	1. Call get_status() 2. Pass validation ID 3. Verify status info	Valid validation ID	Returns status, progress, estimated completion time	Status retrieved	**Pass**
TC-078	Compare multiple validations	Multiple validations exist	1. Call compare_validations() 2. Pass validation IDs 3. Verify comparison	3 validation IDs (comma-separated)	Returns side-by-side comparison with winners per metric	Comparison generated	**Pass**
TC-079	Compare validation versions	Two validations for same idea	1. Call compare_versions() 2. Pass two validation IDs 3. Verify deltas	validation_id_1 (older), validation_id_2 (newer)	Returns score deltas, improvements, declines	Version comparison complete	**Pass**
TC-080	Export validation as JSON	Validation complete	1. Call export_json() 2. Pass validation ID 3. Verify JSON structure	Valid validation ID	Returns complete JSON with all validation data	JSON exported	**Pass**
TC-081	Export validation as PDF with charts	Validation complete	1. Call export_pdf() 2. Generate charts 3. Verify PDF	Valid validation ID	Returns PDF with radar chart, bar chart, all sections	PDF exported	**Pass**
TC-082	Create public share link	Validation exists	1. Call create_share_link() 2. Set privacy=public	Validation ID, privacy_level=public	Returns shareable URL	Share link created	**Pass**

			3. Verify link creation				
TC-083	Create password-protected share link	Validation exists	1. Call create_share_link() 2. Set privacy=password_protected 3. Provide password	Validation ID, privacy_level=password_protected, password="SecurePass123"	Returns protected share link	Protected link created	**Pass**
TC-084	Access shared validation with password	Password-protected link exists	1. Call get_shared_validation() 2. Pass share ID and password 3. Verify access	Share ID, password="SecurePass123"	Returns validation data	Access granted	**Pass**
TC-085	Revoke share link	Share link exists	1. Call revoke_share_link() 2. Pass share ID 3. Verify revocation	Valid share ID	Share link deactivated	Link revoked	**Pass**
TC-086	Test unauthorized access to idea	Two users registered	1. User A creates idea 2. User B tries to access 3. Verify rejection	User A's idea ID, User B's token	Returns HTTP 404 error	Access denied	**Pass**
TC-087	Test unauthorized validation access	Validation exists for User A	1. User B tries to access validation 2. Verify rejection	User A's validation ID, User B's token	Returns HTTP 404 error	Access denied	**Pass**
TC-088	Test expired JWT token	Token issued and expired	1. Wait for token expiration 2. Make authenticated request 3. Verify rejection	Expired JWT token	Returns HTTP 401 Unauthorized	Access denied	**Pass**
TC-089	Test invalid JWT token	Malformed token	1. Create invalid token	Tampered JWT token	Returns HTTP 401 Unauthorized	Access denied	**Pass**

			2. Make authenticated request 3. Verify rejection				
TC-090	Test missing authentication token	Protected endpoint	1. Make request without token 2. Verify rejection	No Authorization header	Returns HTTP 401 Unauthorized	Access denied	**Pass**
TC-091	Test idea creation with missing fields	User authenticated	1. Call create_idea() 2. Pass incomplete data 3. Verify validation error	Title only, missing description	Returns HTTP 422 validation error	Idea not created	**Pass**
TC-092	Test product creation with invalid data	User authenticated	1. Call create_product() 2. Pass invalid data 3. Verify validation error	Empty product name	Returns HTTP 422 validation error	Product not created	**Pass**
TC-093	Test accessing non-existent resource	User authenticated	1. Request resource with invalid ID 2. Verify error response	Non-existent idea ID	Returns HTTP 404 Not Found	Error handled gracefully	**Pass**
TC-094	Test concurrent validation requests	Idea exists	1. Submit multiple validation requests 2. Verify all process 3. Check results	Same idea ID, 3 concurrent requests	All validations process successfully	Multiple validations stored	**Pass**
TC-095	Test password hashing security	User registration	1. Register user 2. Check database 3. Verify hash	Password: "MySecurePass123!"	Password stored as bcrypt hash, not plaintext	Security validated	**Pass**
TC-096	Test weak password rejection	User registration attempt	1. Try to register with weak password 2. Verify rejection	Password: "123"	Returns validation error - password too weak	Weak password rejected	**Pass**

TC-097	Test invalid email format	User registration attempt	1. Try to register with invalid email 2. Verify rejection	Email: "notanemail"	Returns validation error - invalid email format	Invalid email rejected	**Pass**
TC-098	Test duplicate email registration	User already registered	1. Try to register with existing email 2. Verify rejection	Email: "existing@example.com"	Returns HTTP 400 - email already exists	Duplicate prevented	**Pass**
TC-099	Test database connection failure handling	Database unavailable	1. Simulate DB failure 2. Make request 3. Verify error handling	Any API request	Returns HTTP 500 with appropriate error message	Error handled gracefully	**Pass**
TC-100	Test LLM API failure handling	LLM service unavailable	1. Simulate LLM failure 2. Trigger validation 3. Verify error handling	Validation request	Returns appropriate error, validation marked as failed	Error handled gracefully	**Pass**
TC-101	Manually trigger embedding generation	Reddit posts exist, embeddings not generated	1. Call generate_embeddings() 2. Pass input ID 3. Verify generation	Input ID with Reddit posts	Embeddings generated, FAISS index created	Embeddings stored in data/embeddings	**Pass**
TC-102	Get embedding system status	System initialized	1. Call get_system_status() 2. Verify response	No input required	Returns model name, processing mode, default settings	System status retrieved	**Pass**
TC-103	Get embedding details for input	Embeddings exist for input	1. Call get_embedding_details() 2. Pass input ID 3. Verify metadata	Valid input ID	Returns metadata, files list, directory info	Details retrieved	**Pass**
TC-104	Delete embeddings for input	Embeddings exist	1. Call delete_embeddings() 2. Pass input ID 3. Verify deletion	Valid input ID	Embeddings directory deleted	Embeddings removed	**Pass**

TC-105	Get embedding cache statistics	Cache has entries	1. Call get_cache_stats() 2. Verify statistics	No input required	Returns cache hits, misses, hit rate, tiered cache stats	Statistics retrieved	**Pass**
TC-106	Clear embedding cache	Cache has entries	1. Call clear_cache() 2. Specify cache type 3. Verify clearing	cache_type: "all"	Cache cleared, returns embeddings removed and space freed	Cache emptied	**Pass**
TC-107	Test embedding generation with GPU flag	System supports GPU	1. Call generate_embeddings() 2. Set use_gpu=true 3. Verify processing	Input ID, use_gpu=true	Embeddings generated using GPU acceleration	GPU processing verified	**Pass**
TC-108	Test embedding generation with custom batch size	Reddit posts available	1. Call generate_embeddings() 2. Set batch_size=64 3. Verify processing	Input ID, batch_size=64	Embeddings generated with specified batch size	Custom batch size applied	**Pass**
TC-109	List all user embeddings	User has multiple embeddings	1. Call get_user_embeddings() 2. Verify list	User with 3 embedding sets	Returns list of all embeddings with metadata	Embeddings list retrieved	**Pass**
TC-110	Migrate legacy cache to optimized cache	Legacy cache has entries	1. Call migrate_cache() 2. Verify migration 3. Check new cache	Legacy cache with embeddings	Returns migrated count, embeddings moved to optimized cache	Migration complete	**Pass**
TC-111	Manually trigger clustering	Filtered posts available	1. Call cluster_posts() 2. Pass input ID 3. Verify clustering	Input ID with filtered posts	Posts clustered, returns cluster count and statistics	Clusters created	**Pass**
TC-112	Get cluster results by input ID	Clustering completed	1. Call get_cluster_results() 2. Pass input ID 3. Verify results	Valid input ID	Returns clusters, statistics, metadata, file paths	Results retrieved	**Pass**

TC-113	List all cluster results for user	User has multiple clustering results	1. Call list_cluster_results() 2. Verify list	User with 4 clustering results	Returns list of all clustering jobs with metadata	List retrieved	**Pass**
TC-114	Delete cluster results	Cluster results exist	1. Call delete_cluster_results() 2. Pass input ID 3. Verify deletion	Valid input ID	Cluster directory and files deleted	Clusters removed	**Pass**
TC-115	Trigger auto-clustering as background task	Filtered posts available	1. Call auto_cluster() 2. Verify immediate return 3. Check background processing	Valid input ID	Returns immediately, clustering starts in background	Background task initiated	**Pass**
TC-116	Get keywords summary for user	User has keyword generations	1. Call get_keywords_summary() 2. Verify summary format	User with 5 keyword sets	Returns condensed summary with counts and status	Summary retrieved	**Pass**
TC-117	Get keyword generation status	Keywords being generated	1. Call get_keyword_status() 2. Pass input ID 3. Verify status	Valid input ID	Returns processing status, stage, error info	Status retrieved	**Pass**
TC-118	Force regenerate keywords	Keywords already exist	1. Call generate_keywords() 2. Set force_regenerate=true 3. Verify regeneration	Input ID, force_regenerate=true	Old keywords replaced with new generation	Keywords regenerated	**Pass**
TC-119	Auto-generate keywords (convenience endpoint)	User input exists	1. Call auto_generate_keywords() 2. Pass input ID only 3. Verify generation	Valid input ID	Keywords generated without request body	Keywords created	**Pass**
TC-120	Get paginated user keywords	User has 100 keyword sets	1. Call get_user_keywords()	limit=20, skip=40	Returns 20 keywords starting from position 40	Pagination working	**Pass**

			2. Set limit=20, skip=40 3. Verify pagination				
TC-121	Manually trigger pain points ranking	Pain points extracted	1. Call rank_pain_points() 2. Pass input ID 3. Verify ranking	Valid input ID	Pain points ranked with relevance scores	Rankings stored	**Pass**
TC-122	Get ranking results by input ID	Ranking completed	1. Call get_ranking_results() 2. Pass input ID 3. Verify results	Valid input ID	Returns ranked pain points with metadata	Results retrieved	**Pass**
TC-123	Get ranking status	Ranking in progress or completed	1. Call get_ranking_status() 2. Pass input ID 3. Verify status	Valid input ID	Returns processing status, has_results flag, ranking info	Status retrieved	**Pass**
TC-124	Get filtered posts details	Semantic filtering completed	1. Call get_filtered_details() 2. Pass input ID 3. Verify details	Valid input ID	Returns filtered count, similarity stats, sample posts	Details retrieved	**Pass**
TC-125	Trigger auto semantic filtering	Embeddings available	1. Call auto_filter() 2. Pass query and input ID 3. Verify background start	Input ID, query text	Returns immediately, filtering starts in background	Background task initiated	**Pass**
TC-126	Get detailed processing status with stages	Processing in progress	1. Call get_processing_status() 2. Pass input ID 3. Verify stage details	Valid input ID	Returns current stage, progress %, estimated time, stage-by-stage status	Detailed status retrieved	**Pass**
TC-127	Get all processing status for user	User has multiple inputs processing	1. Call get_all_status() 2. Verify list	User with 5 inputs	Returns status for all inputs with stage info	All statuses retrieved	**Pass**
TC-128	Validate JWT token	User logged in with valid token	1. Call validate_token() 2. Pass token 3. Verify validation	Valid JWT token	Returns valid=true with user info	Token validated	**Pass**

TC-129	Update user profile	User authenticated	1. Call update_profile() 2. Pass new full_name and email 3. Verify update	full_name: "John Updated", email: "john.new@example.com"	Profile updated, returns updated user info	Profile saved to database	**Pass**
TC-130	Update profile with duplicate email	Another user has the email	1. Try to update profile 2. Use existing email 3. Verify rejection	Email already registered by another user	Returns HTTP 400 - email already registered	Duplicate prevented	**Pass**